Vectorización de un Kernel Numérico Aplicado a la Detección de Bordes en Imágenes y su Análisis de Rendimiento usando Intel Advisor

Alfonso Pérez García[1]

Resumen— **Se presenta el análisis de rendimiento de cuatro variantes de vectorización SIMD aplicadas a un kernel numérico de detección de bordes mediante el operador Prewitt sobre imágenes en escala de grises de 256×256 píxeles. Las versiones implementadas son: escalar (sin vectorización), vectorización automática del compilador Intel `icpx`, vectorización guiada con `#pragma omp simd simdlen(8)`, y vectorización explícita con intrínsecos AVX2. Los experimentos se ejecutaron con $N = 256 \times 256 \times 800 = 52{,}428{,}800$ operaciones para obtener mediciones estables. El análisis con Intel Advisor revela que el kernel es *memory-bound* con intensidad aritmética teórica $\mathcal{I} = 0.333$ FLOP/Byte, lo que limita las ganancias de vectorización a un máximo de $1.24\times$ sobre la versión escalar (7.34 vs. 5.92 GFLOPS). Los resultados muestran que el cuello de botella no es la capacidad de cómputo SIMD, sino el ancho de banda de memoria disponible, confirmado por la gráfica Roofline de Intel Advisor.**

Palabras clave— **Vectorización SIMD, Intel Advisor, operador Prewitt, detección de bordes, rendimiento HPC, AVX2, análisis Roofline, *memory-bound*.**

I. Introducción

La detección de bordes es una de las operaciones fundamentales en el procesamiento digital de imágenes. Su objetivo consiste en identificar los cambios abruptos en la intensidad luminosa de los píxeles de una imagen, los cuales generalmente corresponden a las fronteras entre objetos, texturas o regiones de interés [1]. Esta operación es el primer paso en cadenas de análisis más complejas, como la segmentación semántica, el reconocimiento de objetos o la reconstrucción tridimensional en aplicaciones de visión artificial, medicina, robótica y teledetección satelital [2].

El rendimiento de estas operaciones se vuelve crítico cuando se procesan grandes volúmenes de información, como video en tiempo real (30–60 cuadros por segundo), imágenes médicas de alta resolución (tomografía, resonancia magnética) o flujos de datos provenientes de sensores en sistemas embebidos. Los procesadores modernos incorporan unidades de ejecución vectorial (SIMD, *Single Instruction, Multiple Data*) capaces de procesar múltiples datos en paralelo dentro de un único núcleo de cómputo. En la arquitectura x86-64, la extensión AVX2 (*Advanced Vector Extensions 2*) permite operar con registros YMM de 256 bits, procesando hasta ocho valores de punto flotante simple (32 bits) en una sola instrucción [3].

Sin embargo, explotar correctamente la vectorización SIMD no es trivial. El compilador puede vectorizar automáticamente cuando detecta patrones seguros, pero en ocasiones requiere ayuda mediante pragmas o directivas. Para máxima eficiencia, el programador puede emplear intrínsecos AVX2 directamente, aunque esto eleva la complejidad del código. Independientemente de la estrategia de vectorización adoptada, el rendimiento final puede estar limitado por el ancho de banda de memoria del sistema, un fenómeno conocido como *memory-bound* [4].

El presente trabajo analiza estas cuatro estrategias aplicadas al kernel numérico del operador Prewitt [1], [5]. Se elige el operador Prewitt frente al Sobel por su estructura con pesos uniformes (± 1), que produce una intensidad aritmética teórica reducida ($\mathcal{I} \approx 0.333$ FLOP/Byte) y lo convierte en un candidato ideal para estudiar el comportamiento *memory-bound* en el modelo Roofline. Se utiliza Intel Advisor como herramienta de perfilado para obtener datos objetivos sobre intensidad aritmética, modelo Roofline y comportamiento de los bucles vectorizados. La hipótesis de trabajo es que la intensidad aritmética del kernel ($\mathcal{I} \approx 0.333$ FLOP/Byte) situará todas las versiones en la zona de memoria limitada del modelo Roofline, independientemente del nivel de vectorización aplicado.

El resto del artículo está organizado de la siguiente manera: la Sección II presenta el marco teórico sobre el operador Prewitt, vectorización SIMD e Intel Advisor; la Sección III describe la metodología experimental; la Sección IV presenta los resultados obtenidos; la Sección V discute e interpreta los datos; y la Sección VI expone las conclusiones del trabajo.

II. Marco Teórico

A. El Operador Prewitt y la Magnitud del Gradiente

El operador Prewitt [1], [5] es un operador diferencial discreto que aproxima el gradiente de la función de intensidad de una imagen en escala de grises. A diferencia del operador Sobel, el Prewitt utiliza pesos uniformes (todos iguales a 1) en sus máscaras de convolución, lo que lo convierte en el más simple de la familia de operadores basados en gradiente.

Las dos máscaras del operador son:

[1]Maestría en Sistemas Distribuidos, Benemérita Universidad Autónoma de Puebla (BUAP), Puebla, Pue., México. Correo: `alfonso.089911@gmail.com`

$$K_x = \begin{pmatrix} -1 & 0 & 1 \\ -1 & 0 & 1 \\ -1 & 0 & 1 \end{pmatrix}, \quad K_y = \begin{pmatrix} +1 & +1 & +1 \\ 0 & 0 & 0 \\ -1 & -1 & -1 \end{pmatrix} \tag{1}$$

donde K_x detecta cambios en la dirección horizontal (bordes verticales) y K_y detecta cambios en la dirección vertical (bordes horizontales). Tras calcular los gradientes G_x e G_y por convolución de la imagen con las máscaras, la magnitud del gradiente en cada píxel se obtiene como:

$$G[i] = \sqrt{G_x[i]^2 + G_y[i]^2} \tag{2}$$

Esta fórmula constituye el *kernel numérico* objeto de este estudio. Para cada elemento i del arreglo de datos se requieren exactamente cuatro operaciones de punto flotante: dos multiplicaciones (G_x^2, G_y^2), una adición y una raíz cuadrada. Por tanto, la intensidad aritmética teórica del kernel, entendida como el cociente entre operaciones de punto flotante (*FLOPs*) y transferencias de memoria (*Bytes*), es:

$$\begin{aligned} \mathcal{I} &= \frac{4 \text{ FLOPs}}{3 \times 4 \text{ Bytes}} = \frac{4}{12} \\ &\approx 0.333 \text{ FLOP/Byte} \end{aligned} \tag{3}$$

donde el denominador refleja la lectura de los dos arreglos de entrada (G_x, G_y) y la escritura del arreglo de salida (*Mag*), cada uno de tipo `float` (4 Bytes).

B. Vectorización SIMD

La vectorización SIMD es una técnica de paralelismo de datos que permite ejecutar la misma operación aritmética sobre múltiples datos simultáneamente utilizando registros vectoriales anchos [6]. En los procesadores Intel modernos existen tres extensiones SIMD principales relevantes para este trabajo:

- **SSE/SSE2** (*Streaming SIMD Extensions*): registros XMM de 128 bits, procesa hasta 4 floats por instrucción.
- **AVX/AVX2** (*Advanced Vector Extensions*): registros YMM de 256 bits, procesa hasta 8 floats por instrucción.
- **AVX-512**: registros ZMM de 512 bits, procesa hasta 16 floats por instrucción.

La vectorización puede obtenerse mediante tres estrategias, en orden creciente de control por parte del programador:

1. **Automática**: el compilador analiza el código escalar y genera instrucciones SIMD cuando es seguro hacerlo. Requiere indicios como el calificador `__restrict` para descartar aliasing de punteros.
2. **Guiada**: el programador añade pragmas o directivas que instruyen al compilador a vectorizar un bucle determinado con un ancho específico. La directiva `#pragma omp simd simdlen(8)` indica al compilador que genere SIMD con 8 elementos por vector.
3. **Explícita**: el programador escribe directamente las instrucciones vectoriales mediante intrínsecos (*intrinsics*), funciones C que se mapean uno a uno con instrucciones ensamblador SIMD. Por ejemplo, `_mm256_sqrt_ps` calcula la raíz cuadrada de 8 floats en paralelo usando el registro YMM.

Un aspecto crucial para que el compilador pueda vectorizar automáticamente un bucle es la ausencia de *aliasing de punteros*. Cuando dos punteros pueden señalar regiones solapadas en memoria, el compilador debe asumir que escribir a través de uno puede modificar el valor leído a través del otro, lo que impide la reordenación de cargas y almacenamientos que la vectorización requiere. El calificador `__restrict` —extensión estándar adoptada por los compiladores C/C++ modernos, incluido `icpx`— permite al programador garantizar explícitamente que los punteros así decorados no comparten ninguna posición de memoria con otros punteros dentro del ámbito de la función. En el kernel Prewitt, los arreglos G_x, G_y y *Mag* son distintos por construcción, de modo que declarar los tres parámetros con `const float* __restrict` y `float* __restrict` es semánticamente correcto y constituye la condición mínima que habilita la vectorización automática en la versión 2 (Automática).

La principal ventaja teórica de la vectorización es un incremento del rendimiento proporcional al ancho del vector: 8× con AVX2 respecto al cómputo escalar. Sin embargo, en la práctica este factor está limitado por el ancho de banda de memoria del sistema [7].

C. Intel Advisor y el Modelo Roofline

Intel Advisor [8] es una herramienta de análisis de rendimiento orientada a la optimización de vectorización y paralelismo. Ofrece dos tipos principales de análisis:

- **Survey**: identifica los bucles del programa, determina si están vectorizados o no, reporta el tiempo de ejecución y el conteo de iteraciones (*trip count*).
- **Tripcounts + FLOP**: mide el número real de operaciones de punto flotante ejecutadas y el volumen de datos transferidos desde/hacia memoria, calculando así la intensidad aritmética medida del programa.

La fase **Tripcounts + FLOP** cuantifica la carga real de trabajo del programa completo. Advisor instrumenta cada bucle del Survey e intercala contadores que registran, durante la ejecución: el número real de iteraciones (*trip count*), las FLOPs ejecutadas —distinguiendo entre operaciones SIMD empaquetadas y escalares— y el volumen de datos transferidos desde cada nivel de la jerarquía de memoria (L1, L2, L3, DRAM). A partir de estos contadores se calcula la **intensidad aritmética medida** del programa según la ec. 4:

$$\mathcal{I}_{\text{medida}} = \frac{\text{FLOPs totales}}{\text{Bytes transferidos a/desde memoria}} \quad (4)$$

Este valor es un promedio del *programa completo*, no del kernel aislado. Las operaciones de I/O de imagen PGM, cálculo de gradientes Prewitt, replicación de datos y escritura del resultado aportan menor densidad aritmética que el kernel de magnitud, diluyendo $\mathcal{I}_{\text{medida}}$ notablemente por debajo del valor teórico del kernel puro. Esta discrepancia es un resultado esperado que el modelo Roofline permite diagnosticar al comparar el punto de operación medido con los techos teóricos del hardware.

El **modelo Roofline** [4] es un marco visual para analizar el rendimiento de un kernel en función de su intensidad aritmética. Define los límites teóricos alcanzables como:

$$P_{\text{máx}} = \text{mín}(P_{\text{peak}},\ \mathcal{I} \times B_{\text{mem}}) \quad (5)$$

donde $P_{\text{máx}}$ es el rendimiento máximo (GFLOPS), P_{peak} es el pico computacional del procesador, $\mathcal{I}$ es la intensidad aritmética (FLOP/Byte) y B_{mem} es el ancho de banda de memoria (GB/s).

Un kernel es *memory-bound* cuando su punto de operación cae a la izquierda del "punto de cresta" (*ridge point*), es decir, cuando la memoria no puede suministrar datos a la velocidad que el procesador puede calcular. En ese caso, incrementar las instrucciones SIMD no mejora el rendimiento, ya que el cuello de botella está en el bus de datos y no en la unidad aritmética.

III. Metodología

A. Descripción del Kernel

El kernel numérico implementado calcula la magnitud del gradiente del operador Prewitt (ec. 2) sobre un arreglo unidimensional de N elementos de tipo `float` (32 bits). Las imágenes de entrada tienen un tamaño de 256×256 píxeles en formato PGM (*Portable Gray Map*) binario (P5), que es el formato de imagen en escala de grises sin compresión más sencillo disponible en el estándar Netpbm.

Para obtener mediciones estables de rendimiento con Intel Advisor se requiere que el número de operaciones sea suficientemente grande [8]. Por ello, el kernel se aplica sobre N_{tot} elementos:

$$N_{\text{tot}} = 256 \times 256 \times 800 = 52{,}428{,}800 \quad (6)$$

Es decir, el kernel se ejecuta como si se procesaran 800 réplicas de la imagen original de forma contigua en memoria, garantizando un número de iteraciones suficiente para que el perfilador de Intel Advisor pueda medir con precisión los trip counts, la intensidad aritmética y el modelo Roofline.

Para asegurar la correcta generación de instrucciones SIMD, los tres arreglos (G_x, G_y y *Mag*) se alojan con alineación a 32 Bytes mediante `_mm_malloc`:

Listing 1: Alocación alineada de arreglos.

```
float* Gx  = (float*)_mm_malloc(N * sizeof(float
    ), 32);
float* Gy  = (float*)_mm_malloc(N * sizeof(float
    ), 32);
float* Mag = (float*)_mm_malloc(N * sizeof(float
    ), 32);
```

El calificador `__restrict` se usa en todas las versiones para indicar al compilador que los punteros no se solapan en memoria (*no aliasing*), condición necesaria para que la vectorización automática sea segura.

B. Plataforma Experimental

Hardware: Procesador Intel de 14 núcleos físicos (gen. Core de última generación), con soporte para extensiones AVX2 y SSE4.2. La arquitectura reportada por Intel Advisor incluye: techo vectorial SP FMA Peak de 148.8 GFLOPS (AVX-512 FMA), techo de adición vectorial SP de 74.55 GFLOPS (AVX2/AVX-512), techo escalar SP de 9.28 GFLOPS, ancho de banda L1 $\approx$ 413 GB/s, ancho de banda L2 $\approx$ 190 GB/s, ancho de banda L3 $\approx$ 65–73 GB/s, y ancho de banda DRAM de 24–27 GB/s.

Software: Compilador Intel `icpx` (Intel oneAPI DPC++/C++ Compiler 2025.3.2) sobre Linux (Fedora 43, kernel 6.18.12, 64 bits). Sistema de profiling: Intel Advisor versión incluida en oneAPI 2025.3.2.

Las opciones de compilación para cada versión se resumen en la Tabla I.

Tabla I: Opciones de compilación por versión.

Versión	Flags de compilación
Escalar	`-g -O2 -no-vec -std=c++17 -fno-inline`
Automática	`-g -O3 -march=native -std=c++17 -fno-inline`
Guiada	`-g -O3 -march=native -qopenmp -std=c++17 -fno-inline`
Explícita	`-g -O3 -march=native -std=c++17 -fno-inline`

La versión Escalar utiliza `-O2` en lugar de `-O3` para evitar que el compilador aplique transformaciones agresivas de bucles —como desenrollado intensivo o *software pipelining*— que podrían mejorar el rendimiento escalar más allá de lo que obtendría un programador típico sin vectorización explícita. El flag `-no-vec` desactiva la vectorización SIMD, mientras que `-O2` preserva las optimizaciones escalares estándar, lo que proporciona la línea base más representativa y conservadora para medir la ganancia real de las estrategias SIMD.

En todas las versiones se añade `-qopt-report=max -qopt-report-phase=vec` para generar el reporte de optimización del compilador (`.optrpt`), que documenta si cada bucle fue vectorizado y con qué ancho vectorial.

C. Versiones Implementadas

Versión 1 — Escalar (línea base): El kernel no utiliza ninguna instrucción SIMD. El flag `-no-vec` prohíbe la vectorización automática al compilador:

Listing 2: Kernel escalar (versión de referencia).

```
void kernel_escalar(
    const float* __restrict Gx,
```

```
    const float* __restrict Gy,
    float* __restrict Mag, int N)
{
    for (int i = 0; i < N; i++)
        Mag[i] = sqrtf(Gx[i]*Gx[i] + Gy[i]*Gy[i
            ]);
}
```

Versión 2 — Automática: Código idéntico al escalar, pero compilado con `-O3 -march=native`. El compilador Intel `icpx` detecta que el bucle es vectorizable (punteros con `__restrict`, sin dependencias entre iteraciones) y genera instrucciones AVX2:

Listing 3: Kernel con vectorización automática.

```
void kernel_auto(
    const float* __restrict Gx,
    const float* __restrict Gy,
    float* __restrict Mag, int N)
{
    for (int i = 0; i < N; i++)
        Mag[i] = sqrtf(Gx[i]*Gx[i] + Gy[i]*Gy[i
            ]);
}
```

Versión 3 — Guiada: Se añade la directiva `#pragma omp simd simdlen(8)` para indicar explícitamente al compilador que vectorice con un ancho de 8 elementos (256 bits, registros YMM):

Listing 4: Kernel con vectorización guiada.

```
void kernel_guiado(
    const float* __restrict Gx,
    const float* __restrict Gy,
    float* __restrict Mag, int N)
{
    #pragma omp simd simdlen(8)
    for (int i = 0; i < N; i++)
        Mag[i] = sqrtf(Gx[i]*Gx[i] + Gy[i]*Gy[i
            ]);
}
```

Versión 4 — Explícita: El programador escribe directamente las instrucciones AVX2 mediante intrínsecos `_mm256_*`. El bucle principal procesa 8 floats por iteración; el bucle residual maneja los elementos sobrantes cuando N no es múltiplo de 8:

Listing 5: Kernel con intrínsecos AVX2 explícitos.

```
void kernel_explicito(
    const float* __restrict Gx,
    const float* __restrict Gy,
    float* __restrict Mag, int N)
{
    int i = 0;
    for (; i <= N - 8; i += 8) {
        __m256 vx  = _mm256_loadu_ps(&Gx[i]);
        __m256 vy  = _mm256_loadu_ps(&Gy[i]);
        __m256 vx2 = _mm256_mul_ps(vx, vx);
        __m256 vy2 = _mm256_mul_ps(vy, vy);
        __m256 sum = _mm256_add_ps(vx2, vy2);
        __m256 res = _mm256_sqrt_ps(sum);
        _mm256_storeu_ps(&Mag[i], res);
    }
    for (; i < N; i++)  // residual escalar
        Mag[i] = sqrtf(Gx[i]*Gx[i] + Gy[i]*Gy[i
            ]);
}
```

D. Procedimiento de Análisis con Intel Advisor

Para cada versión se ejecutaron dos fases de análisis de Intel Advisor desde línea de comandos:

1. **Survey + mezcla de instrucciones estáticas**:

Listing 6: Fase Survey de Intel Advisor.

```
advisor --collect=survey
        --static-instruction-mix
        --project-dir=./adv_X
        -- ./ejecutable
```

2. **Tripcounts + FLOP**:

Listing 7: Fase Tripcounts de Intel Advisor.

```
advisor --collect=tripcounts
        --flop
        --project-dir=./adv_X
        -- ./ejecutable
```

Posteriormente se abrió la interfaz gráfica de Intel Advisor (`advisor-gui`) con los proyectos generados para obtener las gráficas Roofline, los Program Metrics y el Survey de bucles, de los que se extrajeron capturas de pantalla para su análisis.

IV. Resultados

A. Rendimiento del Kernel

La Tabla II resume los resultados cuantitativos obtenidos en la medición directa del tiempo de ejecución del kernel para las cuatro versiones. Los tiempos se midieron desde el propio programa C++ usando `std::chrono::high_resolution_clock` sobre las $N_{\text{tot}} = 52{,}428{,}800$ operaciones. El rendimiento en GFLOPS se calcula como $4 \times N_{\text{tot}}/t_{\text{kernel}}$ y el ancho de banda utilizado como $3 \times 4 \times N_{\text{tot}}/t_{\text{kernel}}$.

Tabla II: Rendimiento de las cuatro versiones del kernel Prewitt.

Versión	Tiempo (s)	GFLOPS	BW (GB/s)	Speedup
Escalar	0.0354	5.92	17.76	1.00×
Automática	0.0286	7.34	22.02	1.24×
Guiada	0.0351	5.97	17.90	1.01×
Explícita	0.0293	7.15	21.44	1.21×

El speedup máximo observado es de 1.24× para la vectorización automática, seguido de 1.21× para la explícita. La versión guiada ofrece prácticamente el mismo rendimiento que la escalar (1.01×), a pesar de estar vectorizada con AVX2. Esto anticipa el comportamiento *memory-bound* que se confirmará en el análisis Roofline.

B. Análisis Survey — Bucles Vectorizados

Las Figuras 1 a 4 muestran la lista de bucles identificados por Intel Advisor para cada versión, con sus tiempos de ejecución y tipo de vectorización detectado.

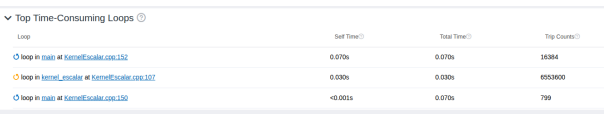
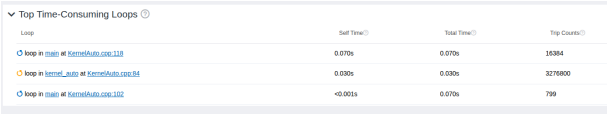


Fig. 1: Survey de bucles — versión Escalar. El kernel (`kernel_escalar:107`) reporta tipo *Scalar* con ISA SSE (uso escalar de registros XMM, sin SIMD empaquetado), conforme al flag `-no-vec`. Trip count: 6,553,600.

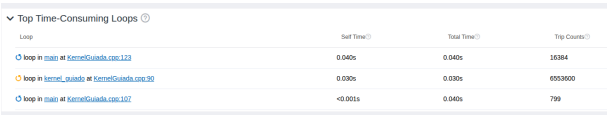
La Tabla III resume los datos del Survey para los kernels principales.



Top Time-Consuming Loops

Loop	Self Time	Total Time	Trip Counts
loop in main at KernelAuto.cpp:118	0.070s	0.070s	16384
loop in kernel_auto at KernelAuto.cpp:84	0.030s	0.030s	3276800
loop in main at KernelAuto.cpp:102	<0.001s	0.070s	799

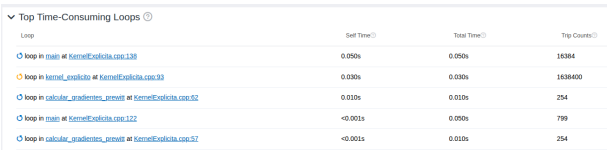
Fig. 2: Survey de bucles — versión Automática. El kernel (`kernel_auto:84`) se marca como *Vectorized (Body)* con ISA AVX2 y trip count de 3,276,800, lo que corresponde a un ancho vectorial de 52,428,800/3,276,800 = 16 floats por iteración.



Top Time-Consuming Loops

Loop	Self Time	Total Time	Trip Counts
loop in main at KernelGuiada.cpp:123	0.040s	0.040s	16384
loop in kernel_guiado at KernelGuiada.cpp:90	0.030s	0.030s	6553600
loop in main at KernelGuiada.cpp:107	<0.001s	0.040s	799

Fig. 3: Survey de bucles — versión Guiada. El kernel (`kernel_guiado:90`) aparece como *Vectorized (Body)* con ISA AVX2 y trip count de 6,553,600, correspondiente a un ancho vectorial de 8 floats, exactamente el valor especificado con `simdlen(8)`.



Top Time-Consuming Loops

Loop	Self Time	Total Time	Trip Counts
loop in main at KernelExplicita.cpp:138	0.050s	0.050s	16384
loop in kernel_explicito at KernelExplicita.cpp:93	0.030s	0.030s	1638400
loop in calcular_gradientes_prewitt at KernelExplicita.cpp:62	0.010s	0.010s	254
loop in main at KernelExplicita.cpp:122	<0.001s	0.050s	799
loop in calcular_gradientes_prewitt at KernelExplicita.cpp:57	<0.001s	0.010s	254

Fig. 4: Survey de bucles — versión Explícita. El kernel (`kernel_explicito:93`) se reporta como *Vectorized (Body)* con ISA AVX2 y trip count de 1,638,400, lo que implica un ancho efectivo de 52,428,800/1,638,400 = 32 floats: el compilador aplicó adicionalmente un desenrollado (*unroll*) de 4× sobre el bucle AVX2 escrito a mano.

Tabla III: Datos del Survey de Intel Advisor para el kernel Prewitt.

Versión	Tipo	ISA	Trip count	Ancho vec.
Escalar	Scalar	SSE	6,553,600**	1
Automática	Vec.(Body)	AVX2	3,276,800	16
Guiada	Vec.(Body)	AVX2	6,553,600	8
Explícita	Vec.(Body)	AVX2	1,638,400	32*

* Unroll 4× del compilador; ancho efectivo = 8 × 4 = 32.

** `-O2` aplica unroll 8× aun con `-no-vec`; Advisor reporta $N/8 = 6{,}553{,}600$.

C. Análisis Roofline

Las Figuras 5 a 8 presentan el modelo Roofline generado por Intel Advisor para cada versión.

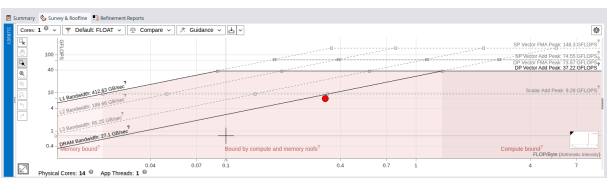


Fig. 5: Modelo Roofline — versión Escalar. El punto rojo se ubica con AI ≈ 0.33 FLOP/Byte y ≈ 6–8 GFLOPS, en la zona acotada entre los techos de ancho de banda L2/L3 y el techo escalar (9.28 GFLOPS).

En los cuatro casos, el punto de operación se ubica en la región denominada "*Bound by compute and memory roofs*", a la izquierda del punto de cresta del Roofline. La intensidad aritmética medida por Advisor para el programa completo es de $\mathcal{I}_{\text{prog}} = 0.100$ FLOP/Byte, inferior al valor teórico

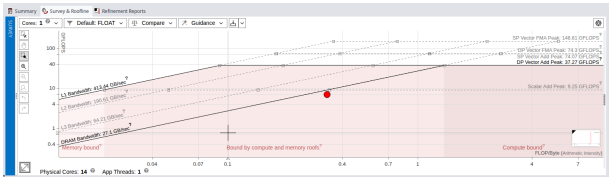


Fig. 6: Modelo Roofline — versión Automática. El punto se sitúa en la misma zona que el Escalar; la ganancia vectorial es mínima porque la memoria no puede alimentar los registros AVX2 a la velocidad máxima.

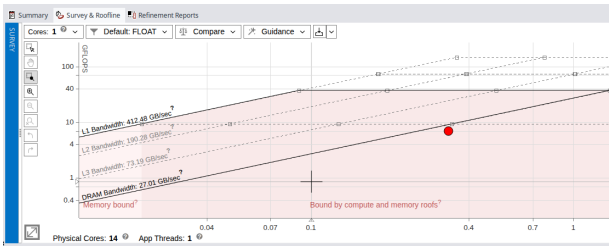


Fig. 7: Modelo Roofline — versión Guiada. Comportamiento idéntico a las versiones anteriores. El ancho vectorial de 8 (menor al de la versión automática) resulta en rendimiento similar al escalar.

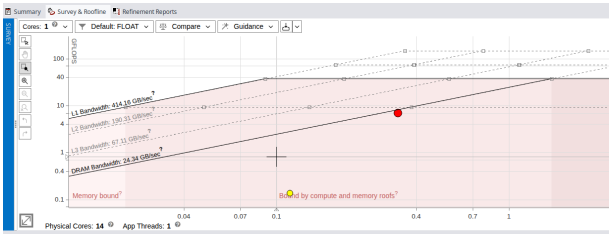
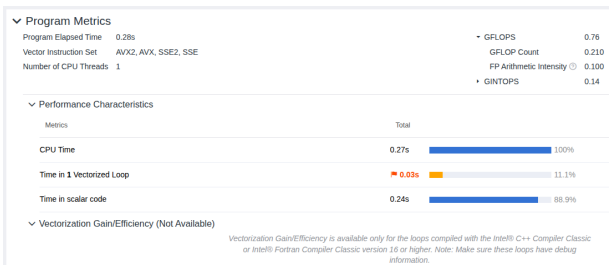


Fig. 8: Modelo Roofline — versión Explícita. A pesar del ancho efectivo de 32 floats (con unroll), el punto permanece en la misma zona *memory-bound*, con rendimiento ligeramente superior al escalar.

del kernel (0.333 FLOP/Byte), porque Advisor incluye todas las operaciones de memoria del programa (I/O de imagen PGM, cálculo de gradientes Prewitt, replicación de datos), que diluyen la intensidad global.

D. Métricas Generales del Programa

Las Figuras 9 y 10 muestran los Program Metrics reportados por Intel Advisor para las versiones Escalar y Automática como versiones representativas.



Program Metrics

Program Elapsed Time	0.28s	GFLOPS	0.76
Vector Instruction Set	AVX2, AVX, SSE2, SSE	GFLOP Count	0.210
Number of CPU Threads	1	FP Arithmetic Intensity	0.100
		GINTOPS	0.14

Performance Characteristics

Metrics	Total	
CPU Time	0.27s	100%
Time in 1 Vectorized Loop	0.03s	11.1%
Time in scalar code	0.24s	88.9%

Vectorization Gain/Efficiency (Not Available)

Vectorization Gain/Efficiency is available only for the loops compiled with the Intel® C++ Compiler Classic or Intel® Fortran Compiler Classic version 16 or higher. Note: Make sure these loops have debug information.

Fig. 9: Program Metrics — versión Escalar. Tiempo total 0.28 s; rendimiento 0.76 GFLOPS; tiempo del bucle vectorizado (SSE escalar) 0.03 s (11 %) frente a 0.24 s (89 %) de código escalar (I/O, gradientes, replicación). ISA del programa: AVX2, AVX, SSE2, SSE.

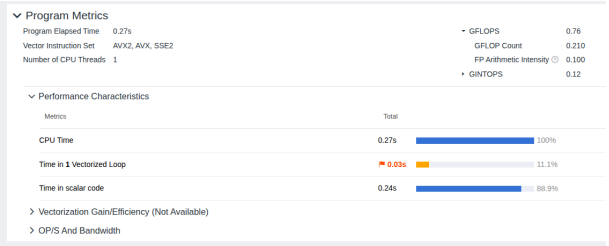


Fig. 10: Program Metrics — versión Automática. Tiempo total 0.27 s; rendimiento 0.76 GFLOPS; tiempo del bucle AVX2 0.03 s (11 %). ISA del programa: AVX2, AVX, SSE2 (sin SSE puro). La distribución de tiempos refleja que el kernel vectorizado es solo una pequeña fracción del programa total.

La Tabla IV consolida los datos de Program Metrics para las cuatro versiones.

Tabla IV: Resumen de métricas de Intel Advisor (Program Metrics).

Versión	Elapsed (s)	GFLOPS	AI (FLOP/B)	Kernel (%)
Escalar	0.28	0.76	0.100	11.1
Automática	0.27	0.76	0.100	11.1
Guiada	0.24	0.86	0.100	12.5
Explícita	0.26	0.82	0.100	12.0

Las Figuras 11 y 12 complementan el panorama para las versiones Guiada y Explícita.

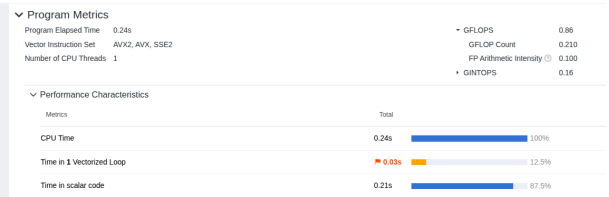


Fig. 11: Program Metrics — versión Guiada. Tiempo total 0.24 s; rendimiento 0.86 GFLOPS; bucle AVX2 con `simdlen(8)` corresponde al 12.5 % del tiempo. Pese a que el ancho vectorial es inferior al de la versión automática, el tiempo total es el menor de las cuatro, por variaciones de timing en el código escalar circundante.

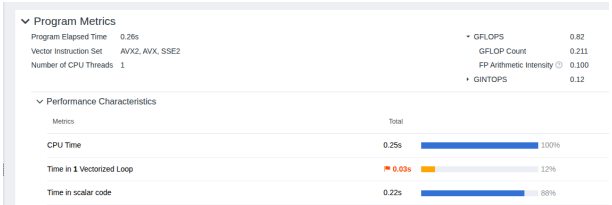


Fig. 12: Program Metrics — versión Explícita. Tiempo total 0.26 s; rendimiento 0.82 GFLOPS; bucle AVX2 el 12 % del tiempo. Dos bucles escalares adicionales de cálculo de gradientes Prewitt (`calcular_gradientes:57`, `:62`) son visibles en el Survey de esta versión; en las demás no superan el umbral de medición de Advisor.

Un hallazgo relevante es que el tiempo total del programa está dominado ($\approx$ 88–89 %) por código escalar: lectura de la imagen PGM, cálculo de gradientes Prewitt con las máscaras 3×3, replicación de datos en memoria y escritura del resultado. El kernel vectorizado representa únicamente el 11–12 % del tiempo de ejecución total. Esta distribución es independiente de la estrategia de vectorización empleada, ya que las partes escalares del programa son las mismas en las cuatro versiones.

E. Análisis de Código Activo (Code Analytics)

El panel Code Analytics de Intel Advisor identifica las funciones que mayor tiempo de cómputo consumen dentro de los bucles vectorizados. En las cuatro versiones, la función más activa reportada es `__intel_avx_rep_memset`, la rutina interna del compilador para inicialización de memoria con instrucciones AVX. Esto confirma que la mayor parte del tiempo vectorizado corresponde a transferencias de datos y no a cómputo aritmético, consistente con el carácter *memory-bound* del kernel. Cabe destacar que las versiones Guiada y Explícita producen perfiles de Code Analytics visualmente idénticos: ambas son kernels AVX2 *memory-bound* que invocan la misma rutina de transferencia, por lo que el *bottleneck* es el mismo independientemente del ancho vectorial (8 vs. 32 floats efectivos). La diferencia entre ambas versiones se aprecia en el Survey (trip count 6,553,600 vs. 1,638,400) y en los GFLOPS (5.97 vs. 7.15 GFLOPS).

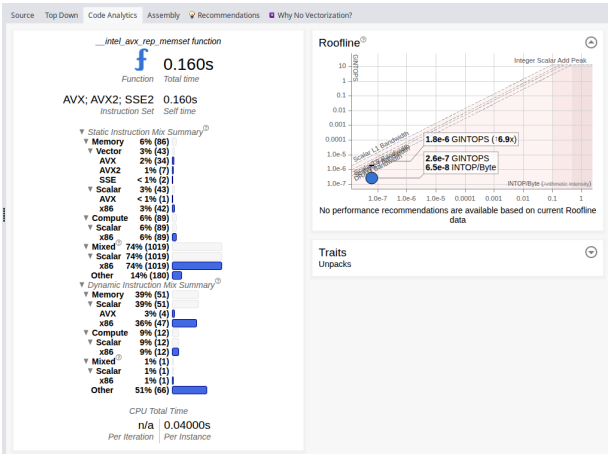


Fig. 13: Code Analytics — versión Escalar. El tiempo total registrado en el bucle es de 0.16 s, con un GINTOPS (giga operaciones enteras por segundo) de speedup 6.9× respecto al techo escalar de referencia. La función dominante es `__intel_avx_rep_memset`, lo que confirma que el tiempo se gasta principalmente en movimiento de datos.

V. Discusión

A. Comparación entre Versiones de Vectorización

Los resultados muestran que la vectorización, aunque presente, produce ganancias modestas sobre el kernel Prewitt en el hardware utilizado. La versión automática es la que mejor rendimiento ofrece (1.24×), seguida de la explícita (1.21×). Resulta llamativo que la vectorización guiada con `simdlen(8)` no mejore significativamente sobre la escalar (1.01×) pese a estar vectorizada con AVX2.

Este comportamiento se explica por el papel del ancho vectorial real. La versión automática utiliza un ancho de 16 floats (AVX2 con posible unroll adicional), procesando el doble de datos por iteración que la guiada (ancho 8). La explícita, aunque nomi-

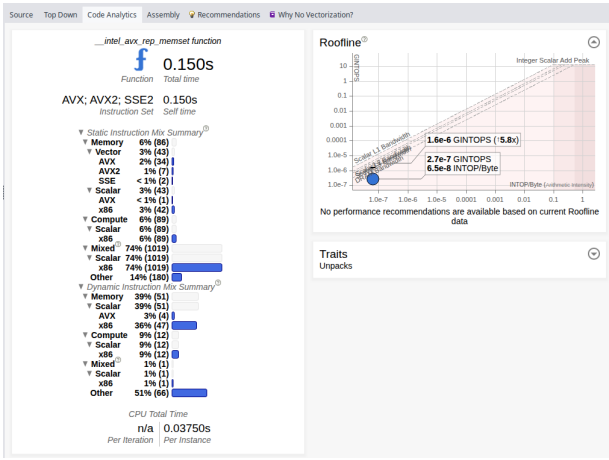


Fig. 14: Code Analytics — versión Explícita. Tiempo de 0.15 s con speedup GINTOPS de 5.8×. Comparando con la versión Escalar (6.9×), la diferencia en el perfil de funciones activas confirma que los intrínsecos AVX2 modifican el patrón de acceso a memoria respecto al código escalar, aunque ambas versiones siguen siendo dominadas por la transferencia de datos.

nalmente de ancho 8, recibe un desenrollado 4× del compilador, resultando en un ancho efectivo de 32, lo que también supera a la guiada. Sin embargo, todos estos anchos vectoriales son incapaces de saturar el ancho de banda de memoria en igual medida que lo harían en un kernel de mayor intensidad aritmética.

Desde la perspectiva de la complejidad de implementación:

- La **automática** requiere el menor esfuerzo y produce el mejor resultado; solo necesita compilar con `-O3 -march=native` y usar `__restrict`.
- La **guiada** requiere conocer el pragma de OpenMP SIMD y elegir el `simdlen` adecuado; resultó inferior porque `simdlen(8)` es más conservador que la decisión automática del compilador.
- La **explícita** requiere dominar los intrínsecos AVX2 y resulta en rendimiento ligeramente menor que la automática, mostrando que el compilador también puede optimizar el código escalar mejor que la versión manual de ancho 8 sin unroll.

B. Interpretación del Modelo Roofline

La intensidad aritmética teórica del kernel es $\mathcal{I} = 0.333$ FLOP/Byte (ec. 3). Este valor sitúa el kernel *por debajo* del punto de cresta del Roofline calculado como:

$$\mathcal{I}_{\text{cresta}} = \frac{P_{\text{peak}}}{B_{\text{mem}}} \approx \frac{9.28 \text{ GFLOPS}}{24 \text{ GB/s}} \approx 0.387 \text{ FLOP/Byte} \tag{7}$$

Al ser $\mathcal{I} = 0.333 < \mathcal{I}_{\text{cresta}} = 0.387$, el kernel es **memory-bound**: el cuello de botella es el ancho de banda de memoria, no la capacidad de cómputo. Esto significa que aunque se utilicen instrucciones AVX2 de máxima eficiencia, los registros YMM permanecerán ociosos una parte del tiempo esperando que la memoria entregue los datos.

El techo alcanzable con la memoria DRAM a 24 GB/s sería:

$$P_{\text{máx}}^{\text{DRAM}} = 0.333 \times 24 \approx 8.0 \text{ GFLOPS} \tag{8}$$

Los resultados experimentales (5.92–7.34 GFLOPS) se sitúan *por debajo* de este límite, confirmando que el kernel opera en la zona de ancho de banda de la caché L2/L3 del procesador (65–190 GB/s). La intensidad aritmética medida por Advisor para el programa completo es $\mathcal{I}_{\text{prog}} = 0.100$ FLOP/Byte; a esa intensidad el techo DRAM equivaldría a $0.100 \times 24 = 2.4$ GFLOPS. Que los valores medidos (5.92–7.34 GFLOPS) superen ampliamente ese techo DRAM confirma que los datos son servidos desde la caché L2/L3 y no desde la DRAM durante la mayor parte del tiempo.

C. ISA Detectada por Versión

Intel Advisor reporta la ISA (*Instruction Set Architecture*) utilizada en cada programa:

- **Escalar**: AVX2, AVX, SSE2, **SSE**. La presencia de instrucciones SSE puras (sin empaquetado) confirma que el kernel usa registros XMM en modo escalar (un solo float por registro), no SIMD empaquetado. El flag `-no-vec` prohíbe el uso vectorial, pero el procesador sigue usando los registros SSE para operaciones escalares.
- **Automática**: AVX2, AVX, SSE2 (sin SSE puro). La ausencia de SSE puro confirma que el kernel escaló al modo AVX2 empaquetado.
- **Guiada**: AVX2, AVX, SSE2 (sin SSE puro). Idéntico al automático en cuanto a ISA, aunque con ancho 8 en vez de 16.
- **Explícita**: AVX2, AVX, SSE2 (sin SSE puro). Los intrínsecos `_mm256_*` garantizan el uso de registros YMM de 256 bits.

D. Dominancia del Código Escalar y Ley de Amdahl

Un resultado relevante es que el bucle de replicación de datos (el que extiende la imagen 800 veces en memoria para alcanzar N_{tot}) consume entre 0.040 y 0.070 s, superando al propio kernel vectorizado (0.030 s). Esto implica que la mayor parte del tiempo lo ocupan operaciones fuera del kernel: lectura del archivo PGM, cálculo de los gradientes Prewitt con las máscaras 3×3 (bucle escalar de 254 iteraciones), replicación de datos y escritura del resultado.

Este comportamiento puede analizarse formalmente con la **Ley de Amdahl** [7]. Sea p la fracción paralelizable del programa y f el factor de aceleración (*speedup*) de esa fracción. El speedup total del programa está acotado por:

$$S(f) = \frac{1}{(1-p) + p/f} \tag{9}$$

En nuestro caso, $p \approx 0.12$ (el kernel representa el 12 % del tiempo total) y $1 - p \approx 0.88$ es la fracción escalar fija. La Tabla V muestra el speedup total máximo teórico en función del factor f del kernel:

Tabla V: Speedup máximo total predicho por la Ley de Amdahl ($p = 0.12$, $1 - p = 0.88$).

Speedup kernel f	**Speedup total** $S(f)$	**Mejora (%)**
1× (escalar)	1.00	—
2×	1.06	+6 %
4×	1.10	+10 %
8× (AVX2 t.)	1.12	+12 %
16× (AVX2 m.)	1.13	+13 %
∞	1.14	+14 %

El speedup máximo del *programa completo*, incluso vectorizando el kernel infinitamente rápido, es de apenas 1.14×. Los speedups anteriores (1.01–1.24×) corresponden *únicamente al kernel*; el speedup del programa completo, estimado con los tiempos totales de Advisor (≈0.24–0.28 s), es de apenas ≈1.03–1.04×, coherente con el techo de Amdahl y con la dominancia del código escalar (88–89 % del tiempo total). La única forma de superar este techo sería vectorizar también las partes escalares del programa (cálculo de gradientes, replicación de datos).

Para superar el límite de 1.14× predicho por Amdahl, es necesario reducir la fracción escalar $1 - p = 0.88$ del programa. Los dos candidatos principales son el bucle de cálculo de gradientes Prewitt (3×3, convolución horizontal y vertical sobre las 254 filas interiores de la imagen) y la rutina de replicación de datos que extiende la imagen 800 veces en memoria para alcanzar N_{tot}. Estos dos bucles operan actualmente en modo escalar a pesar de que sus patrones de acceso son regulares y potencialmente vectorizables. Si el bucle de replicación, que domina el tiempo escalar, se reescribiera con instrucciones AVX2 (`_mm256_store_ps`), y el bucle de gradientes 2D se descompusiera en dos pasadas 1D separables y vectorizadas, la fracción paralela p' aumentaría considerablemente. Formalmente, si se pudiera reducir la fracción escalar de 0.88 a, por ejemplo, 0.30, el techo de Amdahl pasaría de 1.14× a $1/(0.30) \approx 3.3\times$, lo que justifica la vectorización integral del programa y no solo del kernel de magnitud. Este análisis ilustra el principio central de la Ley de Amdahl: una vez que el componente optimizado representa una fracción pequeña del tiempo total, el foco de optimización debe desplazarse hacia las partes que actualmente dominan la ejecución.

E. Limitaciones y Trabajo Futuro

La métrica *Vectorization Gain/Efficiency* de Intel Advisor reporta *Not Available* en todas las versiones. Esto se debe a que dicha métrica requiere el compilador Intel C++ clásico (`icc/icpc` versión 16+), y el presente trabajo utilizó `icpx` (compilador DPC++ de la generación oneAPI). Para trabajo futuro se recomienda explorar: (a) operadores de mayor intensidad aritmética, como el filtro Gaussiano separable. Un filtro Gaussiano 1D de radio k aplicado en dos pasadas (horizontal + vertical) requiere aproximadamente $4(2k + 1)$ FLOPs por píxel frente a 12 Bytes transferidos (2 lecturas + 1 escritura de floats), lo que arroja $\mathcal{I} = (2k + 1)/3$ FLOP/Byte: para $k = 1$ (3×3), $\mathcal{I} \approx 1.0$ FLOP/Byte; para $k = 4$ (9×9), $\mathcal{I} \approx 3.0$ FLOP/Byte. Estos valores superan el punto de cresta del Roofline ($\mathcal{I}_{\text{cresta}} \approx 0.39$ FLOP/Byte con DRAM), situando el filtro Gaussiano en la zona *compute-bound* donde la vectorización AVX2 sí produce ganancias cercanas al factor teórico de 8×, a diferencia del operador Prewitt que permanece en la zona *memory-bound*; (b) análisis multi-hilo con OpenMP para aprovechar los 14 núcleos disponibles; (c) uso de AVX-512 si el hardware lo soporta, para alcanzar anchos de 16 o 32 floats con mayor eficiencia que el unroll.

VI. Conclusiones

Este trabajo presentó el análisis comparativo de cuatro estrategias de vectorización SIMD aplicadas al kernel numérico del operador Prewitt para detección de bordes en imágenes. A partir de los resultados obtenidos se establecen las siguientes conclusiones:

1. **El kernel Prewitt es memory-bound.** Su intensidad aritmética teórica de 0.333 FLOP/Byte lo coloca a la izquierda del punto de cresta del modelo Roofline (0.387 FLOP/Byte), lo que significa que el cuello de botella de rendimiento es el ancho de banda de memoria y no la capacidad de cómputo del procesador. Esta conclusión es independiente del nivel de vectorización aplicado y fue confirmada por las cuatro gráficas Roofline generadas con Intel Advisor.
2. **La vectorización produce ganancias modestas.** El speedup máximo observado fue de 1.24× para la vectorización automática y de 1.21× para la explícita. En un kernel *compute-bound*, estos mismos anchos vectoriales (16× y 32× efectivo) producirían speedups proporcionales al ancho SIMD. La modestia de los resultados es, por tanto, esperada y correcta dado el carácter *memory-bound* del kernel.
3. **La vectorización automática es la más eficiente.** El compilador `icpx` con `-O3 -march=native` elige un ancho vectorial de 16 floats y genera instrucciones AVX2 óptimas sin intervención del programador, superando incluso a la versión explícita en términos de relación rendimiento-esfuerzo. Esto subraya el alto nivel de madurez de los optimizadores modernos de compiladores Intel.
4. **La vectorización guiada requiere ajuste del simdlen.** El pragma `simdlen(8)` impuso un ancho menor al elegido automáticamente por el compilador, resultando en rendimiento prácticamente igual al escalar. Una elección de `simdlen(16)` o dejando que el compilador decida habría producido mejores resultados.
5. **El 88 % del programa es código escalar.** La fracción vectorizable del programa (el kernel) representa solo el 11–12 % del tiempo total de ejecución. El tiempo dominante corresponde a operaciones inherentemente escalares: lectura/escritura de archivos PGM, cálculo de gradientes 2D con máscaras 3×3, y replicación

de datos en memoria. Esta distribución limita el speedup global del programa de acuerdo con la Ley de Amdahl.

6. **Intel Advisor es una herramienta diagnóstica valiosa.** El Survey, los trip counts y el modelo Roofline permitieron confirmar objetivamente el ancho vectorial real de cada versión, identificar el cuello de botella de memoria y cuantificar la fracción vectorizable del programa. Su integración en el flujo de desarrollo es recomendable para cualquier proyecto de optimización de kernels HPC.

Agradecimientos

El autor agradece al Dr. Mario Rossainz López, profesor del curso de Programación en Plataformas Multicore de la Benemérita Universidad Autónoma de Puebla (BUAP), por el diseño de la actividad y la guía metodológica que permitieron llevar a cabo este trabajo de manera rigúrosa.

Referencias

[1] Rafael C. Gonzalez and Richard E. Woods, *Digital Image Processing*, Pearson, New York, NY, 4th edition, 2018, Sección 10.2 "Edge Detection", pp. 704–718.

[2] John Canny, "A computational approach to edge detection," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 8, no. 6, pp. 679–698, 1986.

[3] Intel Corporation, *Intel Intrinsics Guide*, Intel Corporation, 2024, Accedido: febrero 2026.

[4] Samuel Williams, Andrew Waterman, and David Patterson, "Roofline: An insightful visual performance model for multicore architectures," *Communications of the ACM*, vol. 52, no. 4, pp. 65–76, 2009.

[5] Judith M. S. Prewitt, "Object enhancement and extraction," in *Picture Processing and Psychopictorics*, B. S. Lipkin and A. Rosenfeld, Eds., pp. 75–149. Academic Press, New York, 1970.

[6] James Jeffers, James Reinders, and Avinash Sodani, *Intel AVX-512 Instructions and Their Use in Scientific High Performance Computing Applications*, Morgan Kaufmann, Cambridge, MA, 2016.

[7] Herb Sutter, "The free lunch is over: A fundamental turn toward concurrency in software," *Dr. Dobb's Journal*, vol. 30, no. 3, pp. 202–210, 2005.

[8] Intel Corporation, *Intel Advisor User Guide*, Intel Corporation, 2025, Versión incluida en Intel oneAPI Toolkit 2025.3.2. Accedido: febrero 2026.